

PRAYOG INDIA — Technical Architecture

Project: Prayog India | Next.js Web + Flutter Android/iOS + Admin Panel + Store/Walk-in POS +
Inventory & Business Management

Prepared from the Prayog India functional & technical scope PDF.

1. Architecture Overview

Client layer: Next.js customer web, Flutter Android/iOS app, and Store/POS interfaces. Central API backend: NestJS/Node.js REST API.

Data/services: PostgreSQL for business data, Redis for caching, Elasticsearch/OpenSearch for large-catalogue search, object storage for media, CDN/Cloudflare for delivery, plus external payment, shipping, WhatsApp, FCM and email integrations.

2. Recommended Stack

- Web
- Next.js + TypeScript + Tailwind CSS + Shadcn/UI + Framer Motion
- Mobile
- Flutter + Dart
- Backend
- NestJS + Node.js + REST
- Database
- PostgreSQL + Prisma
- Cache
- Redis
- Search
- Elasticsearch/OpenSearch
- Infrastructure
- AWS/VPS + Cloudflare + CDN + Object Storage + backups + monitoring + CI/CD

3. Domain Architecture

Core domains: Identity/RBAC, Stores/Devices, Catalogue, Inventory, Procurement, Orders, Payments, Shipping, Customers/CRM, Coupons/Rewards, Invoices/Documents, Sales/Incentives, Support, CMS, Notifications, Analytics and Audit.

Product catalogue is global. Store inventory is separate by location. A conceptual inventory key is store_id + product_id/variant_id, with orders retaining store and channel attribution.

4. Key Data Entities

- Users, Roles, Permissions, Store Access, Devices
- Stores and authorized store devices
- Products, Variants, Categories, Brands, Attributes, Media, Documents
- Central product catalogue
- Store Inventory, Stock Transactions, Adjustments, Transfers
- Location-specific stock
- Suppliers, Purchases, Purchase Items, Purchase Price History
- Procurement and cost
- Orders, Order Items, Payments, Payment Components

- Commerce/POS
- Shipments, Shipping Rules, Rates
- Logistics
- Customers, Addresses, Wishlists, Reviews, Rewards
- CRM/customer
- Quotations, Proforma Invoices, Tax Invoices
- B2B/sales documents
- Sales Executives, Targets, Incentives, Payouts
- Sales performance
- Tickets, Notifications, CMS, Audit Logs
- Operations/content/security

5. Inventory Architecture

Ranchi: one shared inventory for website, app and Ranchi walk-in. Other stores: independent inventories. Stock transfers are explicit admin-controlled movements.

Online Order → Ranchi reservation; Ranchi Walk-in → Ranchi deduction; Patna Walk-in → Patna deduction; Delhi Walk-in → Delhi deduction.

All stock mutations should be transactional and produce inventory history with product/SKU/variant, location, source channel, order, user/executive, quantity and timestamp.

6. Media & Search

Product images, videos, datasheets and manuals should use cloud/object storage rather than the application server. Store media metadata and storage/CDN references in the database.

Use a dedicated search engine for large catalogues. Redis can cache frequent catalogue and business reads. Use background processing for large imports/media jobs.

7. API & Integration Design

REST APIs should be versioned and organized by domain. Validate DTOs at the API boundary, enforce authorization in backend services, and keep payment/shipping/notification integrations behind service adapters.

Payment lifecycle: create payment intent/order → gateway interaction → backend verification/webhook → transactional order/payment update → invoice/event generation.

Notification events can fan out to FCM, email and WhatsApp according to configured rules.

8. Performance & Deployment

Use SSR/SSG where appropriate, CDN, WebP/AVIF, lazy loading, responsive images, pagination/infinite loading, optimized DB queries, Redis caching and background processing.

Recommended deployment includes automated CI/CD, backups, monitoring, error logging and separate environment configuration for development, staging and production.